A: Request latency, least squares, and ridge

Problem statement and worked resolution

Companion to `A_request_latency.ipynb`

How to use this explanation. Read this alongside the executable cells. The resolution explains both the fitting algorithms and why stable predictions do not always require stable individual coefficients. Section numbers follow the notebook's numbered sections where present. Numerical values are rounded and refer to the recorded run, not universal performance claims.

Problem and notation

Predict request latency y in milliseconds from payload s in MB and queue length q. The simulator generates 600 independent requests with

$$s \sim U(0.5, 10), \quad q \sim \text{Poisson}(3), \quad y = 40 + 8s + 3q + \epsilon, \quad \epsilon \sim N(0, 12^2).$$

All draws above are independent. A second payload measurement is $s' = s + u$, with independent $u \sim N(0, 0.02^2)$. The prime labels a noisy copy, not a derivative. The questions are: can we recover the linear relationship, verify different optimizers, and stabilize coefficients when two inputs carry almost identical information?

An intercept b and slope vector w define $\hat{y} = b + x^T w$. The vector $\beta = (b, w^T)^T$ includes the intercept. In matrix formulas X includes a leading ones column, so $\hat{y} = X\beta$; scikit-learn receives feature columns only and handles its own intercept. Ordinary least squares (OLS) minimizes total squared residuals. A residual is $y_i - \hat{y}_i$. Cross-validation (CV) scores a model on training rows excluded from that particular fit.

1 Generate data and reserve partitions

Use simulation seed 17 and split seed 23. Reserve 120 test rows, then 120 validation rows from the remaining 480; 360 rows remain for training. Validation provides an audit, while five-fold CV within training selects ridge strength. Test rows are used only at the end.

Main Python calls

`np.random.default_rng` supplies reproducible random draws; `train_test_split` separates rows; `DataFrame` preserves feature names and row identifiers. Index-disjointness assertions check partition separation. The duplicated feature is generated without learning anything from targets. Generating it before splitting is safe here; estimating a mean, scale, or model from all rows would not be. The training scatterplot shows the payload relationship, with spread caused by queue variation and outcome noise. It does not reveal an exact deterministic line.

2 Fit a constant and a payload-only line

For a constant prediction, minimizing $\sum_i (y_i - b)^2$ gives $b = \bar{y}$. For the line $b + ws$, differentiate with respect to b and w and solve:

$$w = \frac{\sum_i (s_i - \bar{s})(y_i - \bar{y})}{\sum_i (s_i - \bar{s})^2}, \qquad b = \bar{y} - w\bar{s}.$$

The denominator is nonzero because payload varies. The implementation computes this formula and compares it with `LinearRegression().fit(...)` using `assert_allclose`.

The recorded baseline is 88.998 ms; the fitted line is

$$\hat{y} = 45.849 + 8.524s.$$

The slope means an additional predicted 8.524 ms per MB. It need not equal the generating slope 8 in a finite sample. Since queue is omitted, the intercept also absorbs its average contribution; it should not simply be compared with 40 as though the models had the same inputs. In the population, independence gives $\mathbb{E}[y \mid s] = 49 + 8s$.

3 Fit both features using matrices

With columns $(1, s, q)$, solve

$$\min_{\beta} \|X\beta - y\|^2.$$

Setting the gradient to zero yields the normal equations $X^T X\beta = X^T y$. These equations explain the solution, but explicitly forming $(X^T X)^{-1}$ is an unnecessary and potentially unstable computational step. The notebook uses `np.linalg.lstsq` and verifies agreement with `LinearRegression`.

Parameter	Generating value	Fitted value
Intercept (ms)	40	36.2358
Payload (ms/MB)	8	8.4116
Queue (ms/request)	3	3.6336

Each slope now describes a change with the other input held fixed. The numerical solvers agree; discrepancies from generating coefficients are statistical estimation error, not evidence that a solver failed.

4 Solve the same problem by gradient descent

For n fitting rows, define

$$J(\beta) = \frac{1}{2n}\|X\beta - y\|^2, \qquad \nabla J(\beta) = \frac{1}{n}X^T(X\beta - y).$$

Starting at zero, update $\beta \leftarrow \beta - \eta\nabla J$. The Hessian is $X^T X/n$, so a safe step is $\eta = 1/L$ with $L = \|X\|_2^2/n$; the matrix norm is the largest singular value. Stop when the gradient norm is below 10^{-8}, with a 20,000-update limit.

Main Python calls

`least_squares_gd` is the notebook's implementation; `np.linalg.norm(..., ord=2)` computes the spectral norm. `StandardScaler.fit` learns training means μ_j and standard deviations a_j; `transform` applies $z_j = (x_j - \mu_j)/a_j$. Scaling improves numerical conditioning without changing the set of linear prediction functions. If the standardized coefficients are $(\tilde{b}, \tilde{w})$, recover original-unit coefficients using

$$w_j = \tilde{w}_j/a_j, \qquad b = \tilde{b} - \sum_j w_j\mu_j.$$

Both runs agree with least squares within roughly 10^{-7} ms in training predictions. Raw features require 5,919 updates, whereas standardized features require 9. These are iteration counts for the stated stopping criterion, not a universal speed ratio.

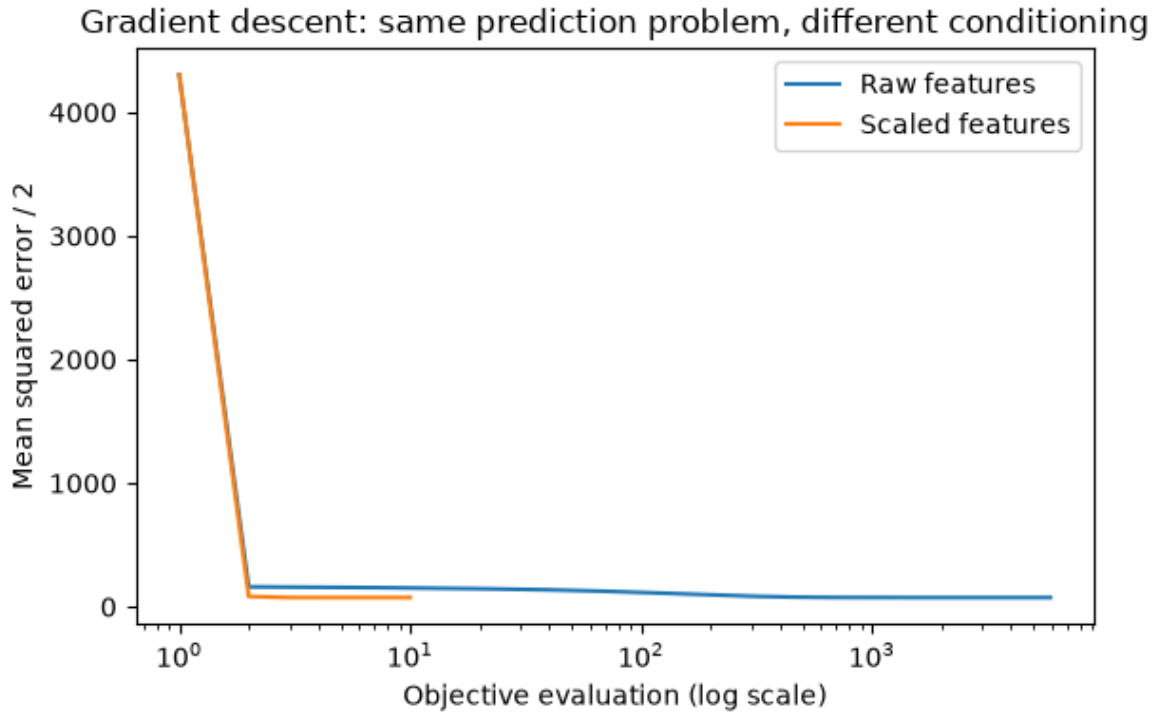


Figure 1: The two optimizations reach the same objective. Standardization changes the geometry and greatly reduces the update count in this example.

5 Choose ridge strength on the expanded features

With (s, q, s'), two columns are nearly equal. Changes to the payload coefficients can offset each other, producing little prediction change. This makes individual OLS slopes poorly determined. Ridge instead minimizes

$$\frac{1}{2n}\|X\beta - y\|^2 + \frac{\lambda}{2}\|w\|^2,$$

where w contains the three standardized slopes and excludes the intercept. The penalty discourages large compensating coefficients. Scaling matters for ridge because the penalty depends on coefficient units.

Main Python calls

`KFold` supplies five fitting/scoring partitions; `Ridge(alpha=...)` fits penalized least squares. Its objective is RSS plus `alpha` times the squared slope norm, so multiplying our objective by $2n$ gives `alpha = n_fit * lambda`. Search $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10\}$. Each fold fits on 288 rows and scores on 72. Fit a fresh scaler on the 288 rows, transform both subsets with that scaler, fit ridge, and score held-out RMSE. Reuse the same folds for all penalties. Average fold RMSE values; an exact tie uses grid order. Fold standard deviation describes score variability, not a confidence interval.

The selected λ is 10^{-4}, with CV RMSE about 12.183 ms. Very strong shrinkage, $\lambda = 10$, gives about 22.886 ms. Refit the scaler and model on all 360 training rows; the final library penalty is $360 \times 10^{-4} = 0.036$. Also fit expanded OLS as a prespecified comparison. Do not refit on validation after inspecting it.

6 Inspect shrinkage and bootstrap stability

The ridge path plots absolute standardized slopes as the penalty varies. Absolute values hide signs, so this plot illustrates magnitudes, not the direction of every association. Under correlated inputs, individual coefficient paths need not decrease monotonically.

A bootstrap sample draws 360 training rows with replacement. Across 100 resamples, refit both scaler and model, convert slopes back to original units, and predict at $(s, q, s') = (5, 3, 5)$. Keep the chosen λ fixed: this assesses fitting variability at that setting, not the entire selection procedure.

Bootstrap standard deviation	OLS	Ridge
Payload slope (ms/MB)	33.914	7.668
Copy slope (ms/MB)	33.930	7.683
Sum of payload slopes (ms/MB)	0.235	0.234
Fixed-request prediction (ms)	0.641	0.630

The individual coefficients fluctuate substantially, but their sum is stable. Indeed,

$$w_s s + w_{s'} s' = (w_s + w_{s'})s + w_{s'} u.$$

When u is small, predictions largely depend on the sum. Ridge reduces individual coefficient variability without dramatically changing prediction variability at this typical input. This does not justify prediction for inputs with s' far from s.

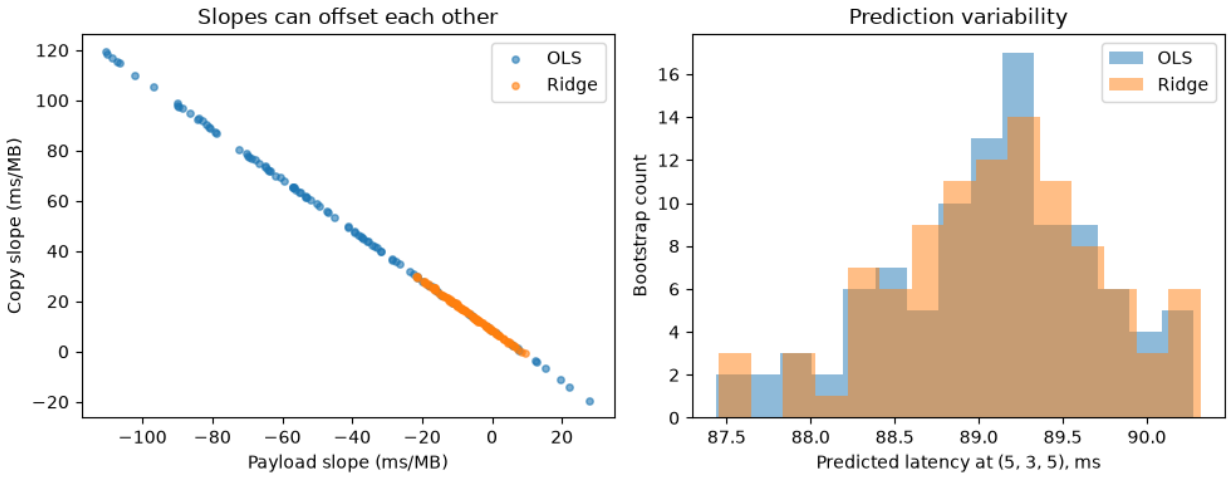


Figure 2: Large opposite changes in the two OLS payload slopes can cancel. The fixed-input prediction distributions are much tighter than the separate coefficient distributions suggest.

7 Residuals and validation audit

Training residuals from the two-feature model should have no systematic trend if the mean relationship is well captured. A curved pattern can indicate a missing relationship; changing spread can indicate varying noise. Training residuals are optimistic for evaluating new-row prediction. Queue and payload are independent in this generator, so omitting queue need not produce an obvious residual trend against payload. A lack of visible trend is not proof that all useful inputs have been included. `regression_report` predicts with each frozen model and computes mean

absolute error (MAE) and RMSE:

$$\text{MAE} = \frac{1}{m}\sum_i |y_i - \hat{y}_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{m}\sum_i (y_i - \hat{y}_i)^2}.$$

Both are in milliseconds. The validation audit leaves all choices unchanged; it gives approximately 12.505 ms RMSE for the two-feature model and 12.519 for expanded ridge.

8 Final test and answers to the interpretation questions

Frozen model	Test MAE (ms)	Test RMSE (ms)
Training mean	21.702	25.810
Payload only	10.427	12.466
Payload plus queue	10.299	12.737
Expanded OLS	10.321	12.738
Expanded ridge	10.300	12.729

The simple payload-only model happens to have the lowest test RMSE, while the two-feature model has lower MAE. Different losses weight errors differently, and a 120-row test sample need not reproduce population rankings. Do not use this table to repeat model selection.

The key resolutions are: all three numerical approaches agree; standardization improves optimization; collinearity can destabilize coefficients more than predictions; and ridge stabilizes the duplicated slopes without guaranteeing a large improvement in held-out error. The independent noise has standard deviation 12 ms, so even the exact conditional mean has population RMSE 12 ms. A finite test score may lie above or below it.

Further explanation: why these results occur

The geometry of a nearly duplicated feature

Imagine first that $s' = s$ exactly. Then $w_s s + w_{s'} s' = (w_s + w_{s'})s$: the data identify the sum, but cannot distinguish $(w_s, w_{s'}) = (8, 0)$ from $(108, -100)$. This is a lack of identifiability, not a failure of Python. With a small difference $u = s' - s$, these alternatives become distinguishable only through the tiny term $w_{s'}u$. Noise in the targets can then produce large changes in the two separate fitted slopes.

Ridge makes an additional preference explicit. For a fixed sum $a = w_s + w_{s'}$, the identity

$$w_s^2 + w_{s'}^2 = \frac{a^2}{2} + \frac{(w_s - w_{s'})^2}{2}$$

shows why its penalty prefers balanced coefficients when the two columns are equal and identically scaled. With a near-copy and a weak selected penalty they need not be equal or even both positive. Thus the negative mean payload slope in the bootstrap table does not by itself imply that larger payloads reduce latency.

Why the same redundancy also affects optimization

For centered, standardized feature matrix Z, the ridge normal equations are

$$(Z^T Z + n\lambda I)w = Z^T(y - \bar{y}\mathbf{1}).$$

If $Z^T Z$ has a very small eigenvalue d, unpenalized fitting divides the corresponding target projection by d. Ridge replaces that divisor by $d + n\lambda$, preventing enormous amplification in that direction. This explains both numerical and statistical stabilization. It does not remove the underlying ambiguity about separate real-world effects.

For gradient descent on a quadratic, coefficient error in an eigenvector direction is multiplied by $1 - \eta d/n$ per update. When eigenvalues have very different sizes, a step small enough for the largest one makes slow progress in the smallest direction. Standardization often improves this imbalance, but it does not generally remove collinearity. Scaling and ridge solve related but different problems.

Prediction uncertainty is not just coefficient uncertainty

The bootstrap fixed-request distribution measures variability in the estimated conditional mean. A future observed latency also contains new noise. Conditional on the training data, if the true mean is $f(x)$,

$$\mathbb{E}[(Y_{\text{new}} - \hat{f}(x))^2 \mid x, \text{training}] = 12^2 + (f(x) - \hat{f}(x))^2.$$

The cross-term vanishes because future noise is independent and mean zero. Consequently a bootstrap prediction standard deviation near 0.63 ms does not imply that future latencies lie within roughly 0.63 ms of the prediction. The irreducible noise is much larger.

Documentation details that change how code should be read

The LinearRegression reference separates constructor parameters from fitted attributes and methods. In particular, `fit_intercept=True` means the library supplies the intercept, and `score` returns R^2, not RMSE. The Ridge reference states its summed-loss objective; that is why our average-loss λ requires the factor n. The trailing underscore in `coef_` means an attribute learned during fitting.

Practical checks. Keep original feature units in your reported coefficients; preserve column order; reuse the training scaler for predictions; and inspect predictions as well as coefficients when collinearity is present. A solver tolerance tests optimization accuracy, not correctness of the statistical model. If the selected penalty lies at the edge of a grid, a wider search may be a sensible *new development experiment*, but the existing final test must not be repeatedly reused as an untouched assessment.